

<u>CL-2006</u> <u>Operating System Lab</u>	<u>LAB -04</u> <u>System Call related to Process</u> <u>Management</u>
---	---

National University of Computer and Emerging Science
Spring 2026

Table of Content

Objective.....	3
Introduction.....	3
System Call.....	3
1. Communication with the Kernel:.....	3
2. Transition to Kernel Mode:.....	3
3. Requesting Operating System Services:.....	4
4. Implementation by the Kernel:.....	4
5. Return Values and Error Handling:.....	4
C Program Structure for doing System Calls.....	5
Process Creation.....	6
Information Maintenance.....	7
File Management.....	7
Lab Activity.....	8

Objective

After completing this lab, you will be able to:

- Demonstrate System Calls Programmatically
- Develop an understanding of system calls by implementing various functions programmatically and explore the functionality and usage of each system call through practical coding exercises, emphasizing their role in process management, information retrieval, and file manipulation.
- Implement system calls such as `fork()`, `wait()`, `exit()`, `exec()`, `sleep()`, `getpid()`, `getppid()`, `alarm()`, `open()`, `read()`, `write()`, and `close()` to perform essential operating system tasks.

Introduction

Modern operating systems follow a layered design. User programs do not directly access hardware or critical system resources. Instead, they request services from the operating system kernel using **system calls**. This lab introduces system calls through hands-on programming in C, helping students understand how user programs interact with the kernel.

System Call

A system call is a mechanism that allows a user-level program to request a service or operation from the kernel (the core part of the operating system). These services can range from low-level tasks like accessing hardware devices to higher-level operations like creating or terminating processes.

1. Communication with the Kernel:

The kernel is the central component of an operating system responsible for managing hardware resources and providing essential services to user programs. User programs, including applications and utilities, operate in a restricted environment known as **user mode**. Direct access to hardware resources is prohibited in user mode for security and stability reasons.

2. Transition to Kernel Mode:

When a user program requires a service that only the operating system kernel can provide, it needs to transition from user mode to a more privileged mode called **kernel mode**. System calls provide a controlled mechanism for this transition. When a system call is invoked, the program switches from user mode to kernel mode to execute the requested operation.

3. Requesting Operating System Services:

System calls act as an interface between user programs and the operating system kernel. They allow user programs to request various services or operations provided by the kernel.

Examples of common system calls include:

- Opening, reading from, writing to, and closing files.
- Creating, executing, and terminating processes.
- Allocating and managing memory.
- Interacting with hardware devices such as printers and network interfaces.

4. Implementation by the Kernel:

Each system call has a unique identifier or number associated with it, known as a syscall number. This number is used by the kernel to identify the requested service. The kernel contains implementations for handling different system calls. When a system call is invoked, the kernel executes the corresponding routine to perform the requested operation.

5. Return Values and Error Handling:

System calls typically return a value to the calling program, indicating the success or failure of the requested operation. If an error occurs during the execution of a system call, the kernel returns a special error code, allowing the user program to handle the error gracefully.

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Management	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()

Figure 1: System Calls for Windows and Unix/Linux Operating Systems

C Program Structure for doing System Calls

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

int main () {

    // Declare variables and necessary data structures
    // Make explicit system calls
    // Handle errors if any
    // Perform necessary operations using system calls

    return 0;
}
```

This basic structure provides a framework for writing C programs that utilize explicit Linux/UNIX system calls, enabling developers to interact directly with the underlying operating system for performing various tasks. Explanation of various part of the programs follows:

Library	Description
#include <stdio.h>	The standard input-output library for basic input and output operations.
#include <stdlib.h>	The standard library for general utilities and memory allocation.
#include <unistd.h>	The POSIX (Portable Operating System Interface) standard header file for various system calls, such as process management (fork(), exec(), exit(), etc.).
#include <sys/types.h>	Definitions for various data types used in system calls, such as pid_t for process IDs.
#include <sys/wait.h>	Definitions for functions related to process management and waiting for child processes to terminate.
#include <errno.h>	Definitions for error numbers that may occur during system calls.

Process Creation

A **process** is a program in execution. Each process has its Process ID (PID), Parent Process ID (PPID), Its own address space and execution state The processes in most systems can execute concurrently, and they may be created and deleted dynamically. Thus, these systems must provide a mechanism for process creation and termination.

Function	Syntax	Description	Library	Possible Values
fork()	pid_t fork();	Creates a new child process	<unistd.h>	Returns 0 (child process), >0 (parent process with child PID), -1 (error)
getpid()	pid_t getpid();	Returns the process ID (PID) of the calling process	<unistd.h>	Returns the PID of the current process
getppid()	Pid_t getppid();	Returns the parent process ID(PPID) of the calling process	<unistd.h>	Returns the PID of the parent process
execvp()	Int execvp(const char *file, const char *arg, ..., NULL);	Replaces the current process image with a new program	<unistd.h>	file: Name of executable arg: Command-line arguments NULL: Must be the last argument
wait()	pid_t wait(int *status);	Waits for a child process to terminate	<sys/wait.h>	status: Pointer to an integer storing exit status Returns child's PID or -1 on error
WIFEXITED()	Int WIFEXITED(int status);	Checks if child exited normally	<sys/wait.h>	Returns nonzero if child terminated normally
WEXITSTATUS()	Int WEXITSTATUS(int status);	Retrieves exit status of child process	<sys/wait.h>	Returns the exit status (0 for success, nonzero for failure)
perror()	void perror(const char *msg);	Prints an error msg for the last system call error	<stdio.h>	msg: Custom error message before system error description
exit()	void exit(int status);	Terminate the process with given status	<stdlib.h>	status: EXIT_SUCCESS (0) or EXIT_FAILURE (non - zero)

Information Maintenance

Information maintenance system calls allow a running process to query the operating system about itself or its execution environment. These system calls are essential because they help in process identification and management, enable debugging and monitoring, support security checks and allow time-based control of program execution. They are widely used in system utilities, servers, shells, and process schedulers.

Function	Syntax	Description	Library	Possible Values
getpid();	pid_t getpid();	Returns the process ID (PID) of the calling process	<unistd.h>	Returns the PID of the current process
getppid();	pid_t getppid();	Returns the parent process ID (PPID) of the calling process	<unistd.h>	Returns the PID of the parent process
getuid();	uid_t getuid();	Returns the user ID (UID) of the calling process	<unistd.h>	Returns the UID of the user executing the process
sleep();	int sleep();	Process goes into an inactive state for a time period	<stdlib.h>	Sleep(2*1000)
alarm();	Int alarm(n);	Used to set an alarm timer for a process.	<stdlib.h>	alarm(20)

File Management

File management system calls allow processes to create, open, read, write, and close files. Files are treated as streams of bytes, and the OS manages storage, access permissions, file positioning and data consistency.

“In UNIX/Linux, everything is treated as a file — including devices.”

Function	Syntax	Description	Library	Possible Values
open()	int open(const char *pathname, int flags, mode_t mode);	Opens a file with the specified flags mode	<fcntl.h>	Flags: O_RDONLY (Read only), O_WRONLY (Write only), O_RDWR (Read & Write), O_CREAT (Create file if it does not exist), O_TRUNC (Truncate file), O_APPEND (Append mode) Modes: S_IRUSR (User read), S_IWUSR (User write),

				S_IXUSR (User execute), S_IRGRP (Group read), S_IWGRP (Group write), S_IXGRP (Group execute), S_IROTH (Others read), S_IWOTH (Others write), S_IXOTH (Others execute)
read()	ssize_t read(int fd, void *buf, size_t count);	Reads data from a file descriptor into a buffer	<unistd.h>	fd: File descriptor returned by open() buf: Pointer to buffer where data will be stored count: Number of bytes to read
write()	ssize_t write(int fd, const void *buf, size_t count);	Writes data from a buffer to a file descriptor	<unistd.h>	fd: File descriptor returned by open() buf: Pointer to buffer containing data to write count: Number of bytes to write
close()	int close(int fd);	Closes a file descriptor	<unistd.h>	fd: File descriptor to close (returned by open())
perror()	void perror(const char *msg);	Prints an error message for the last system call error	<stdio.h>	msg: Custom error message to print before system error description
exit()	void exit(int status);	Terminates the program with a given status	<stdlib.h>	status: EXIT_SUCCESS (0) or EXIT_FAILURE (non-zero)

Lab Activity

- Write a program that accomplish the following purpose:
 - Call the system call to create the child process and store the value returned from the call.
 - A. If the returned value is less than zero,
 - I. Print "Unsuccessful Child Process Creation".
 - II. Terminate using exit system call
 - B. If the return value is greater than zero
 - I. Add a wait system call so that the parent would wait for the child process to complete.
 - II. Make a loop that prints even numbers from 1 - 10
 - III. Print "Parent Ends"
 - C. If the return value is equal to zero
 - I. Print the parent ID
 - II. Make a loop that prints odd numbers from 1 - 10
 - III. Print "Child Ends"
 - Stop

2. Write a Program that Creates n-child process from same parent process using fork() in C
3. Write Program to create four processes (1 parent and 3 children) where they terminates in a sequence as follows:
 - Parent process terminates at last
 - The first child terminates before the parent and after the second child.
 - The second child terminates after the last and before the first child.
 - The third child terminates first.
4. Write a program which creates 4 processes for parallel programming. Each parent will wait for the termination of its child.
5. Implement the following 8 tree structures. Each node must print its name and PID.
e.g. I am Process A and my PID is 2453

